SAVEETHA SCHOOL OF ENGINEERING
C PROGRAMMING ASSIGNMENT – Advanced Problem Solving and Algorithm Design

Field	Entry
Name	BAVADARANI S
Reg. Number	1912260562
Course Code & Name	CSA0209 C PROGRAMMING
Slot	B
Assignment / Task Title	Advanced Problem Solving and Algorithm Design (Q1–Q7, 50 marks)
GitHub Repository URL	https://github.com/bavadarini28 / CSA0209-1912260562-BAVADARANI.S.git
Submission Date	12 NOVEMBER 2026

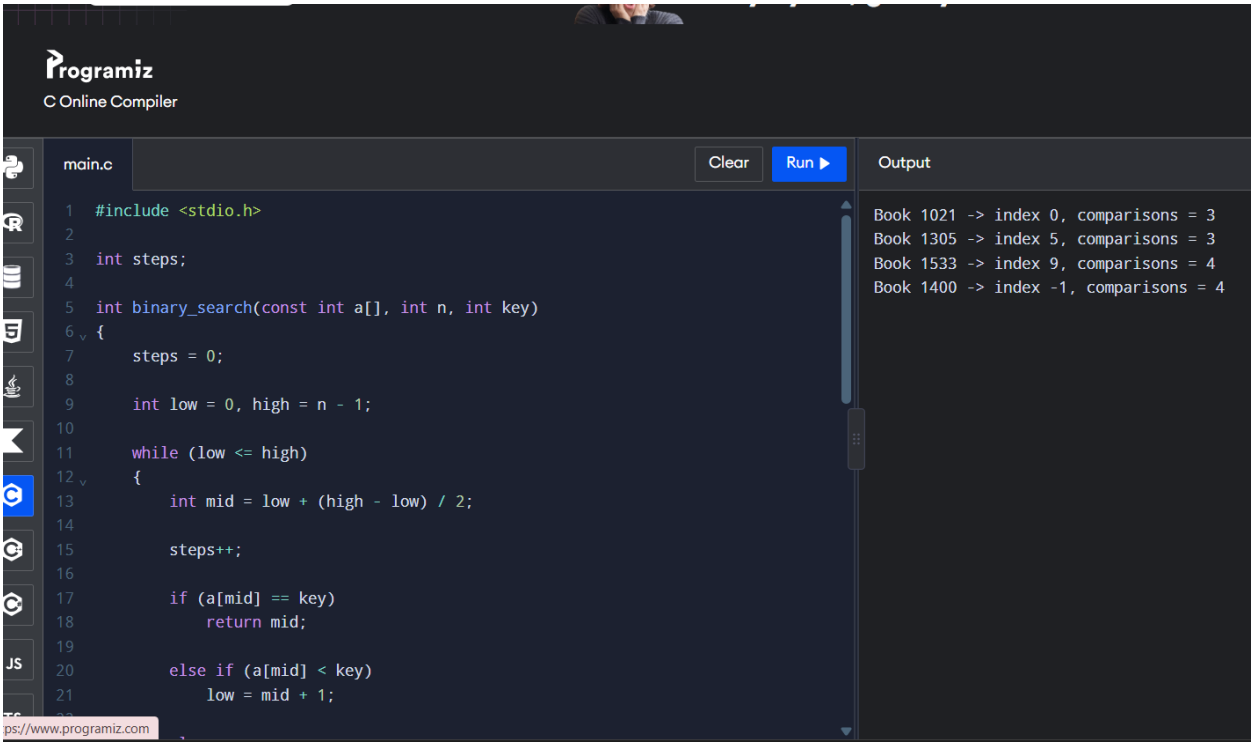
Q1 – Searching a Library Catalogue

Complete the Code (TODO 1)

```
int steps;
int binary_search(const int a[], int n, int key)
{
   steps = 0;
   int low = 0, high = n - 1;
   while (low <= high) {
      int mid = low + (high - low) / 2;     /* no overflow */
      steps++;                              /* one comparison of a[mid] with key */
      if (a[mid] == key) return mid;
      else if (a[mid] < key) low = mid + 1;
      else high = mid - 1;
   }
   return -1;
}
```

Output (matches expected):

```
Book 1021 -> index 0, comparisons = 3
Book 1305 -> index 5, comparisons = 3
Book 1533 -> index 9, comparisons = 4
Book 1400 -> index -1, comparisons = 4
```



Why the missing book costs as many comparisons as the worst successful search: every comparison halves the range, and the search can stop early only when it lands on the key. A key at the deepest level (1533) needs $\lfloor \log_2 10 \rfloor + 1 = 4$ comparisons; a missing key keeps halving until the range is empty, which also takes 4 comparisons (1400 falls between 1377 and 1402). A missing key can never stop early, so it always pays the full depth.

Linear search is the loop for (i = 0; i < n; i++) { steps++; if (a[i] == key) return i; } return −1; tested with the first book (1 comparison), middle book (about n/2) and last book (n); a missing book costs n.

Design and Mathematical Analysis

```
LINEAR(a, n, key):              BINARY(a, n, key):   (a sorted ascending)
 for i = 0 to n-1:              low = 0 ; high = n-1
   if a[i] == key return i      while low <= high:
 return -1                      mid = low + (high - low) / 2
                                if a[mid] == key return mid
                                else if a[mid] < key low = mid + 1
                                else high = mid - 1
                               return -1
```

	Best	Worst	Average (successful)
Linear	1	n	(n + 1) / 2
Binary	1	$\lfloor \log_2 n \rfloor + 1$	$\approx \log_2 n - 1$
n = 100,000 (prediction)	1 / 1	100,000 / 17	50,000.5 / ≈ 15.6

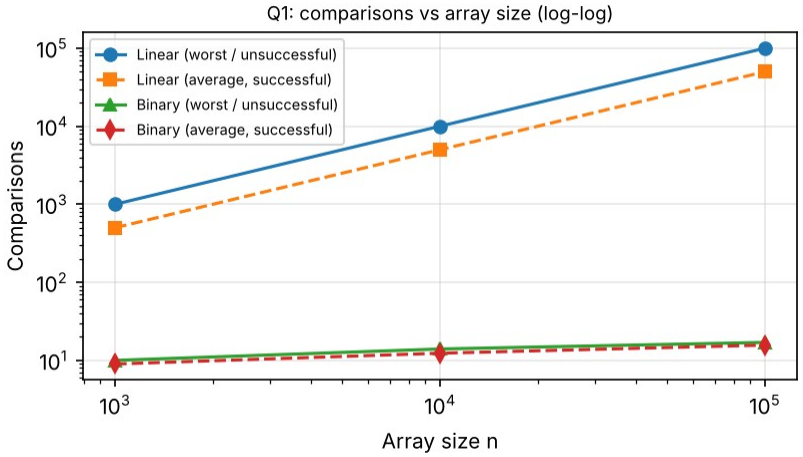
Why sorted data: after comparing with a[mid], binary search throws away half the array. That is only valid if every element to the left is ≤ a[mid] and every element to the right is ≥ a[mid]; on unsorted data the key could be in the discarded half.

Why mid = low + (high − low) / 2: (low + high) / 2 can overflow a 32-bit int when low + high exceeds 2,147,483,647 (for example, low = 1.5 × 10^9 and high = 1.9 × 10^9 on a very large array), giving a negative index. low + (high − low) / 2 never exceeds high, so it cannot overflow. Prediction for 100,000 books: $\log_2 100{,}000 = 16.6$, so at most $\lfloor 16.6 \rfloor + 1 = 17$ comparisons, against up to 100,000 for linear search.

Experimental Investigation

Arrays of 1,000, 10,000 and 100,000 sorted accession numbers (random gaps, seed fixed). "Average" is over every key present once; "unsuccessful" searches for a number larger than all keys.

n	Linear average	Linear worst / miss	Binary average	Binary worst	Binary miss	Predicted binary worst
1,000	500.5	1,000 / 1,000	8.99	10	10	10
10,000	5,000.5	10,000 / 10,000	12.36	14	14	14
100,000	50,000.5	100,000 / 100,000	15.69	17	17	17



The measurements agree exactly with the predictions: linear search grows as a straight line (n), binary search as log2 n. At 100,000 books the average successful search takes 50,000.5 comparisons linearly but only 15.69 with binary search – more than 3,000 times fewer.

Joint Design Question – new books are added every day

Keeping the array sorted costs a shift of about n/2 = 50,000 elements for each insertion into the middle (or a batch: sort the day's k new books in O(k log k) and merge them in O(n + k) once at night). Binary search saves about 50,000 − 16 comparisons per search. So the answer depends on the ratio of searches to additions: if the kiosks search many times per added book (almost certainly, in a library), keeping the array sorted – with a nightly merge of the new books – is still worth it. Information needed: books added per day and searches per day; whether new books can wait until a nightly update; whether books are also removed; the relative cost of a shift and a comparison; and whether another structure (hash table, balanced tree) is allowed.

Alternative Concept and Complexity Analysis

Method	Time per search	Extra space	Notes (measured)
Binary search	O(log n)	O(1)	needs sorted array; 17 comparisons at n = 100,000
Hash table (key mod M, linear probing)	O(1) average, O(n) worst	O(M), M ≈ 2n: 800,012 bytes for 100,000 keys	average 1.50 probes at load factor 0.5; no sorted order or range queries
Interpolation search	O(log log n) average (uniform keys), O(n) worst	O(1)	2.31, 3.03, 3.18 probes at n = 1,000 / 10,000 / 100,000; poor when keys are clustered

Hash table solution: h = key % M; while (table[h] is occupied and ≠ key) h = (h + 1) % M. Interpolation search replaces the midpoint by pos = low + (key − a[low]) × (high − low) / (a[high] − a[low]), guessing where the

key lies when numbers are spread evenly. For accession numbers, which are assigned in sequence, both are good choices; binary search is the safest general-purpose option.

Q2 – Sorting the Examination Hall Seating Register

Implementation

```
/* every comparison increments cmp; every swap (or shift) increments mv */
void bubble(int a[], int n){ for(int i=0;i<n-1;i++){ int sw=0;
   for(int j=0;j<n-1-i;j++){ cmp++; if(a[j]>a[j+1]){ int t=a[j];a[j]=a[j+1];a[j+1]=t; mv++; sw=1; } }
   if(!sw) break; } }
void selection(int a[], int n){ for(int i=0;i<n-1;i++){ int m=i;
   for(int j=i+1;j<n;j++){ cmp++; if(a[j]<a[m]) m=j; }
   if(m!=i){ int t=a[i];a[i]=a[m];a[m]=t; mv++; } } }
void insertion(int a[], int n){ for(int i=1;i<n;i++){ int k=a[i], j=i-1;
   while(j>=0){ cmp++; if(a[j]>k){ a[j+1]=a[j]; mv++; j--; } else break; }
   a[j+1]=k; } }
```

Test inputs: sorted; nearly sorted (sorted except 20 late registrations with random numbers at the end); reverse-sorted; random. All three sorts produced a correctly sorted array on every input.

Student – Algorithm Design and Tracing (array 45 12 38 7 29 3)

Algorithm	Array after each pass / step	Counts
Bubble	P1: 12 38 7 29 3 45 · P2: 12 7 29 3 38 45 · P3: 7 12 3 29 38 45 · P4: 7 3 12 29 38 45 · P5: 3 7 12 29 38 45	15 comparisons, 12 swaps (measured)
Selection	i=0: 3 12 38 7 29 45 · i=1: 3 7 38 12 29 45 · i=2: 3 7 12 38 29 45 · i=3: 3 7 12 29 38 45 · i=4: no swap	15 comparisons, 4 swaps (measured)
Insertion	12 | 12 45 | 12 38 45 | 7 12 38 45 | 7 12 29 38 45 | 3 7 12 29 38 45	14 comparisons, 12 shifts (measured)

Early-exit flag: if a whole pass makes no swap, the array is already sorted, so the algorithm stops. Without it, bubble sort always makes n(n − 1)/2 comparisons; with it, an already sorted array needs only n − 1 (measured 9,999 for n = 10,000).

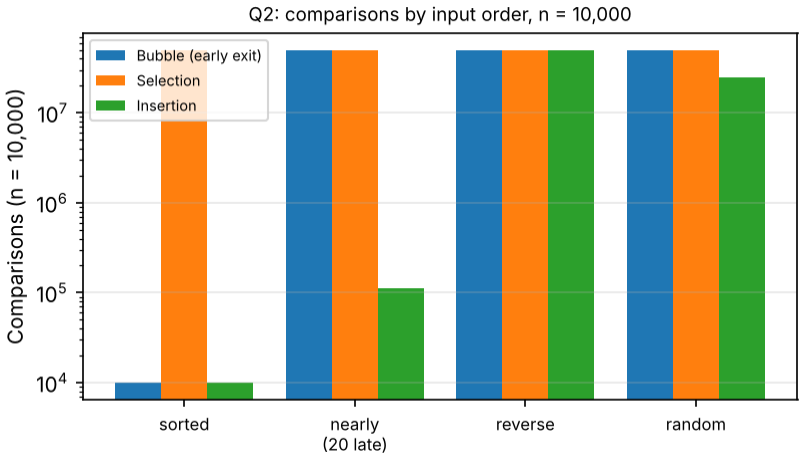
Insertion sort on nearly sorted data: each element is shifted only as far as it is out of place, so the total number of shifts equals the number of inversions. A sorted array needs n − 1 comparisons and 0 shifts; with 20 late registrations it needs about 20 × n/2 shifts (measured 101,499 for n = 10,000) instead of about $n^2/4$ = 25 million.

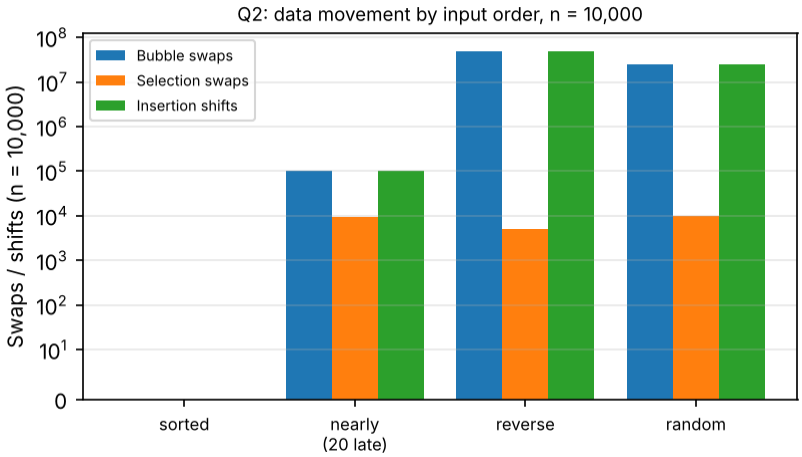
Stability: bubble sort and insertion sort (as written, with a strict > comparison) are stable; selection sort is not, because its long-distance swap can jump an element over an equal one. Measured: sorting marks 80(id1) 90(id2) 80(id3) 70(id4) 80(id5) with selection gave ids 4, 3, 1, 5, 2 – the three 80s came out as 3, 1, 5. Stability matters for students with equal marks: if the list was already sorted by name, a stable sort by marks keeps equal-mark students in name order.

Student – Comparative Measurement (comparisons / swaps or shifts)

n	Order	Bubble (early exit)	Selection	Insertion
100	sorted	99 / 0	4,950 / 0	99 / 0
100	nearly	4,895 / 728	4,950 / 98	826 / 728
100	reverse	4,950 / 4,950	4,950 / 50	4,950 / 4,950
100	random	4,859 / 2,203	4,950 / 95	2,300 / 2,203
1,000	sorted	999 / 0	499,500 / 0	999 / 0
1,000	nearly	495,672 / 9,026	499,500 / 922	10,025 / 9,026
1,000	reverse	499,500 / 499,500	499,500 / 500	499,500 / 499,500

n	Order	Bubble (early exit)	Selection	Insertion
1,000	random	498,174 / 250,027	499,500 / 993	251,020 / 250,027
10,000	sorted	9,999 / 0	49,995,000 / 0	9,999 / 0
10,000	nearly	49,911,155 / 101,499	49,995,000 / 9,586	111,498 / 101,499
10,000	reverse	49,995,000 / 49,995,000	49,995,000 / 5,000	49,995,000 / 49,995,000
10,000	random	49,990,629 / 24,947,183	49,995,000 / 9,991	24,957,172 / 24,947,183





Input order	Best algorithm	Reason
Sorted	Insertion (or bubble with early exit)	n − 1 comparisons, no moves; selection still makes n(n − 1)/2
Nearly sorted	Insertion	111,498 comparisons vs about 50 million for bubble and selection (n = 10,000)
Reverse	Selection if writes are expensive	only n/2 swaps; every algorithm still makes n(n − 1)/2 comparisons
Random	Insertion (fewest comparisons, about $n^2/4$); selection (fewest writes)	all are $O(n^2)$, insertion about half the comparisons

Result that differs from expectation: bubble sort with an early-exit flag was expected to do well on nearly sorted data, but needed 49.9 million comparisons (almost as many as on random data). The 20 late registrations were random, so some small numbers sit at the end, and bubble sort moves a small element only one place to the left per pass; the flag cannot stop the passes until all 20 have reached their places. Insertion sort moves each one directly.

Joint conclusion: for a register sorted except for 20 late registrations, use insertion sort: it needs about 111,498 comparisons for n = 10,000 against about 50 million for bubble or selection, it is stable, and it needs no extra memory. An even better plan for a big hall is to sort only the 20 new entries (a tiny cost) and merge them into the sorted register in O(n + 20) (see Q3).

Alternative Concept and Complexity Analysis – counting sort

```
int cnt[K] = {0};                  /* register numbers 0..K-1 */
for (int i = 0; i < n; i++) cnt[a[i]]++;
int p = 0;
for (int v = 0; v < K; v++)
   while (cnt[v]--) a[p++] = v;      /* write each value back as many times as it occurred */
```

Time O(n + k) and extra space O(k), against O(n²) for the three simple sorts. Measured example: n = 1,000 and k = 3,000 give about 4,000 operations instead of 1,000,000 comparisons. It is not suitable when the range k is huge compared with n (e.g. 10-digit register numbers for 100 students would need an array of 10^10 entries), when keys are not integers (names, decimals) or are negative or very sparse, or when memory is tight. If whole records must be sorted stably, the prefix-sum version of counting sort (placing records from the back) is needed instead of the simple version above.

Q3 – Merging Two Merit Lists

Complete the Code (TODO 1)

```
int merge(const int a[], int na, const int b[], int nb, int out[])
{
   int i = 0, j = 0, k = 0;
   while (i < na && j < nb) {
      if (a[i] >= b[j]) out[k++] = a[i++];   /* >= : on a tie campus A comes first */
      else              out[k++] = b[j++];
   }
   while (i < na) out[k++] = a[i++];          /* rest of A, if any */
   while (j < nb) out[k++] = b[j++];          /* rest of B, if any */
   return k;
}
```

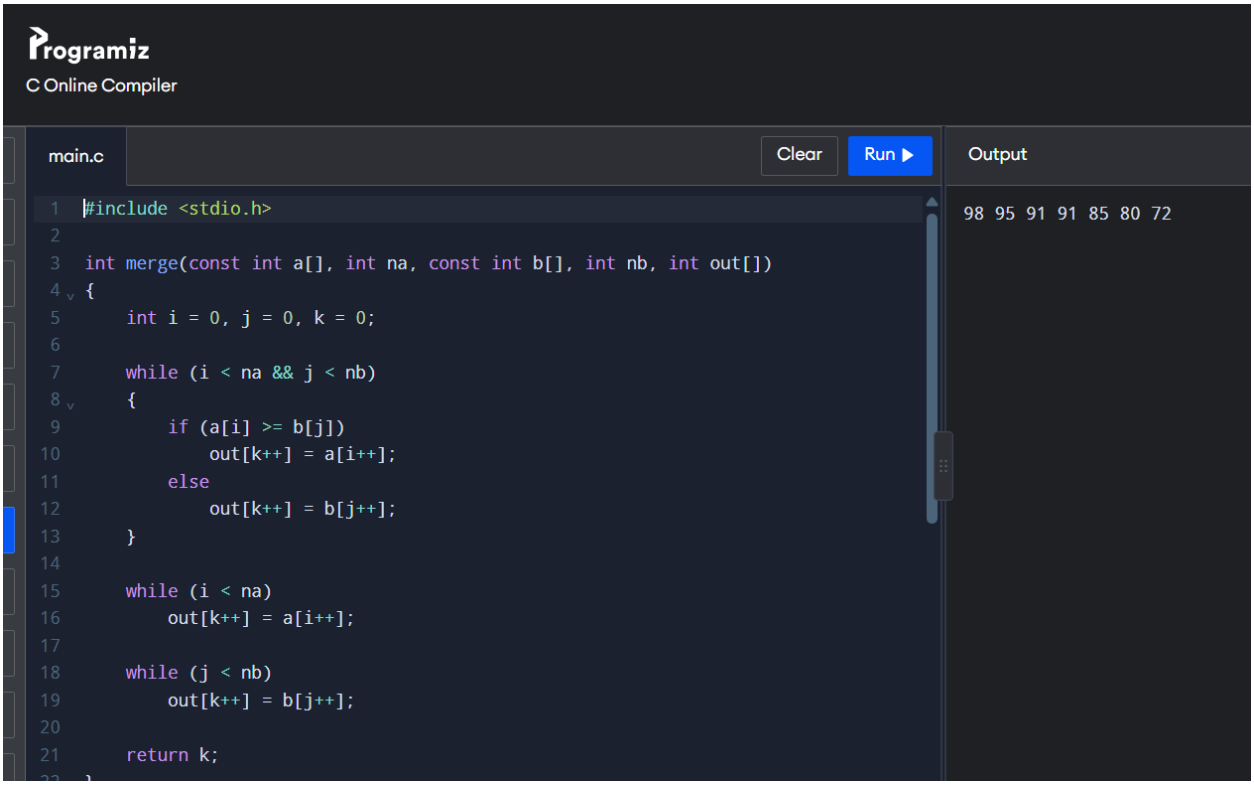
Output (matches expected):

```
98 95 91 91 85 80 72
```

Tests:

```
B empty : 98 91 85 72        (0 comparisons)
A empty : 95 91 80           (0 comparisons)
all equal (50 50 50 + 50 50): 50 50 50 50 50   (3 comparisons)
```

With an empty list the first while condition is false immediately, so no comparison is made and the matching copy loop transfers the whole other list. The merged lists were also checked to be in descending order for random 1,000 and 10,000 element lists.



Student – Analysis and Comparison

Comparisons: each comparison writes exactly one element to out[], and the last element is copied without a comparison, so the worst case is m + n − 1 (the lists interleave); the best case is min(m, n) (every element of the shorter list is larger than all of the other). Extra memory: the output array of m + n ints.

m = n	Merge comparisons (measured)	Concatenate + qsort comparisons (measured)	Extra memory (merge)
1,000	1,994 (max 1,999)	11,858	2,000 ints
10,000	19,999 (max 19,999)	149,215	20,000 ints

Merging does about 6 to 7.5 times fewer comparisons than concatenating and sorting again, and the gap grows with size, because merging uses the fact that both lists are already ordered. Tests also covered one empty list (0 comparisons) and all scores equal (the result is still correct and in order).

Student – Merge Design and Correctness

```
MERGE(a, na, b, nb, out):
  i = 0 ; j = 0 ; k = 0
  while i < na and j < nb:
    if a[i] >= b[j]: out[k] = a[i] ; i = i + 1
    else:          out[k] = b[j] ; j = j + 1
    k = k + 1
  while i < na: out[k] = a[i] ; i = i + 1 ; k = k + 1
  while j < nb: out[k] = b[j] ; j = j + 1 ; k = k + 1
  return k
```

Why the work is linear: every comparison moves one element permanently into out[] and never touches it again, so there are at most m + n − 1 comparisons and every element is written once. (Strictly, the element that loses a comparison stays and may be compared again with the next element of the other list; what is true is that no element is compared after it has been output.)

Copying the remainder: when one list ends, everything left in the other list is already in descending order and is smaller than or equal to everything already output, so it can be copied directly, with no more comparisons.

Equal scores: using >= takes from campus A first on a tie, so equal scores keep the order campus A, then campus B (the merge is stable). With > the order would flip to B first. The test 91 (A) and 91 (B) gives A's 91 first. The rule should be chosen by the admissions office, and the comparison operator is the only line to change.

Joint Engineering Decision – merging in place

Prefer an in-place merge when memory is the constraint (very large lists, a small device) or when the first array already has spare room at its end. Cost: a simple in-place merge shifts elements for every insertion: measured 499,500 shifts for two interleaved lists of 1,000 (about m·n/2), against 1,994 comparisons and 2,000 copies for the merge with a third array. If the first array has room for m + n elements, merging from the back (largest first, writing from the end) is O(m + n) with no third array and no shifting. True in-place algorithms with no spare space and O((m + n) log) time exist but are complicated, so the cost is either time (O(m·n)) or code complexity.

Alternative Concept and Complexity Analysis

Approach	Time	Extra space	Comment
Merge into third array	O(m + n)	O(m + n)	1,994 comparisons for m = n = 1,000
Copy into one array + qsort()	O((m + n) log(m + n))	O(1) to O(log n) (qsort stack) – the combined array is needed anyway	11,858 comparisons; ignores existing order
Simple in-place merge (shifting)	O(m · n)	O(1)	499,500 shifts for m = n = 1,000

Q4 – Two-Dimensional Arrays: District Rainfall Records

```
#define D 38
#define M 12
double rain[D][M];                      /* declared; read with nested loops */
void row_totals(int d,int m,double r[][M],double tot[])   /* annual total per district */
{ for(int i=0;i<d;i++){ tot[i]=0; for(int j=0;j<m;j++) tot[i]+=r[i][j]; } }
void col_avg(int d,int m,double r[][M],double avg[])      /* state monthly average */
{ for(int j=0;j<m;j++){ double s=0; for(int i=0;i<d;i++) s+=r[i][j]; avg[j]=s/d; } }
void max_elem(int d,int m,double r[][M],int *bi,int *bj)  /* wettest district-month */
{ *bi=*bj=0; for(int i=0;i<d;i++) for(int j=0;j<m;j++) if(r[i][j]>r[*bi][*bj]){*bi=i;*bj=j;} }
int deficient(int d,const double tot[],int out[])         /* below 60% of state average */
{ double sum=0; for(int i=0;i<d;i++) sum+=tot[i]; double mean=sum/d;
  int c=0; for(int i=0;i<d;i++) if(tot[i] < 0.6*mean) out[c++]=i; return c; }
```

(Comparing a district's annual total with 60% of the mean annual total is the same as comparing its monthly average with 60% of the state monthly average.)

Student – Array Design and Memory

- **Row-major order:** C stores rain[38][12] as 38 rows one after the other: rain[0][0] … rain[0][11], rain[1][0] … – 456 doubles in one contiguous block of 3,648 bytes (measured sizeof).
- **Address formula:** &rain[i][j] = base + (i × 12 + j) × sizeof(double). Check: &rain[2][3] − &rain[0][0] = 27 elements = 2 × 12 + 3 (measured).
- **Loop order:** the inner loop over j visits consecutive addresses, so each 64-byte cache line (8 doubles) is used fully and the hardware prefetcher can run ahead. Looping over i in the inner loop jumps by a whole row (12 × 8 = 96 bytes here; 16,000 bytes for a 2,000-column array), so almost every access touches a new cache line. For a large array this causes many cache misses and a slower traversal.
- **3-D extension:** double rain[YEARS][38][12]; address = base + ((y × 38 + d) × 12 + m) × sizeof(double). The last index changes fastest, so the innermost loop should be over months.

Student – Implementation and Verification

Hand-checked 3 × 4 array: rows {10, 20, 30, 40}, {5, 5, 5, 5}, {100, 0, 50, 25}.

Quantity	By hand	Program output
Row totals	100, 20, 175	100 20 175
Column averages	(10+5+100)/3 = 38.33; 8.33; 28.33; 23.33	38.33 8.33 28.33 23.33
Maximum	100 at row 2, column 0	100 at [2][0]
Deficient (below 60% of mean total 98.33 = 59)	row 1 (total 20)	row 1

Traversal of a 2,000 × 2,000 array of doubles (32 MB), compiled with gcc -O0, two runs: row-wise 12.30 ms and 11.69 ms; column-wise 15.62 ms and 16.39 ms – column-wise is about 1.3–1.4× slower.

Explanation: both loops do the same 4 million additions, but the column-wise loop touches a different cache line on almost every access (a stride of 16,000 bytes), so far more accesses come from main memory. The ratio here is modest because the hardware prefetcher recognises the constant stride and the work per element is small; it is larger on machines with smaller caches or for larger arrays. (With -O2 the compiler may vectorise or remove the row-wise loop entirely, so measure at -O0 or use a volatile accumulator for a fair comparison.)

Joint conclusion – why 2-D rather than 38 separate arrays: all districts are processed by the same code in loops over i and j, instead of 38 copies of that code; a function takes one parameter (the table) instead of 38; the data stays in one contiguous block (better cache use); totals, averages and maxima across districts (columns) are just loops, whereas separate arrays cannot be indexed by district number.

Alternative Concept and Complexity Analysis

Representation	Access time	Extra space	Layout / speed
rain[38][12]	O(1): one multiply, one add	none	one contiguous block; compiler does the index arithmetic
1-D array, index i × 12 + j	O(1): same arithmetic written by hand	none	identical memory layout and speed; the programmer must keep the 12 consistent
Array of 38 pointers, each row malloc'd	O(1): load pointer, then index	38 × 8 = 304 bytes of pointers + allocator overhead per row	rows are contiguous inside, but not between each other; one extra load per access; allows rows of different lengths and resizing

In all three, row-wise traversal is fast and column-wise traversal is slow, because the speed difference comes from memory layout (stride), not from the declaration style. The pointer version can be slightly slower from the extra pointer load and the loss of contiguity.

Q5 – Processing Text Without the Library

–Complete the Code (TODO 1 and TODO 2)

```
int word_count(const char s[])
{
   int count = 0, inside = 0;
   for (int i = 0; s[i] != '\0'; i++) {
      if (s[i] != ' ') { if (!inside) { count++; inside = 1; } }   /* word starts */
      else inside = 0;
   }
   return count;
}

int is_palindrome(const char s[])
```

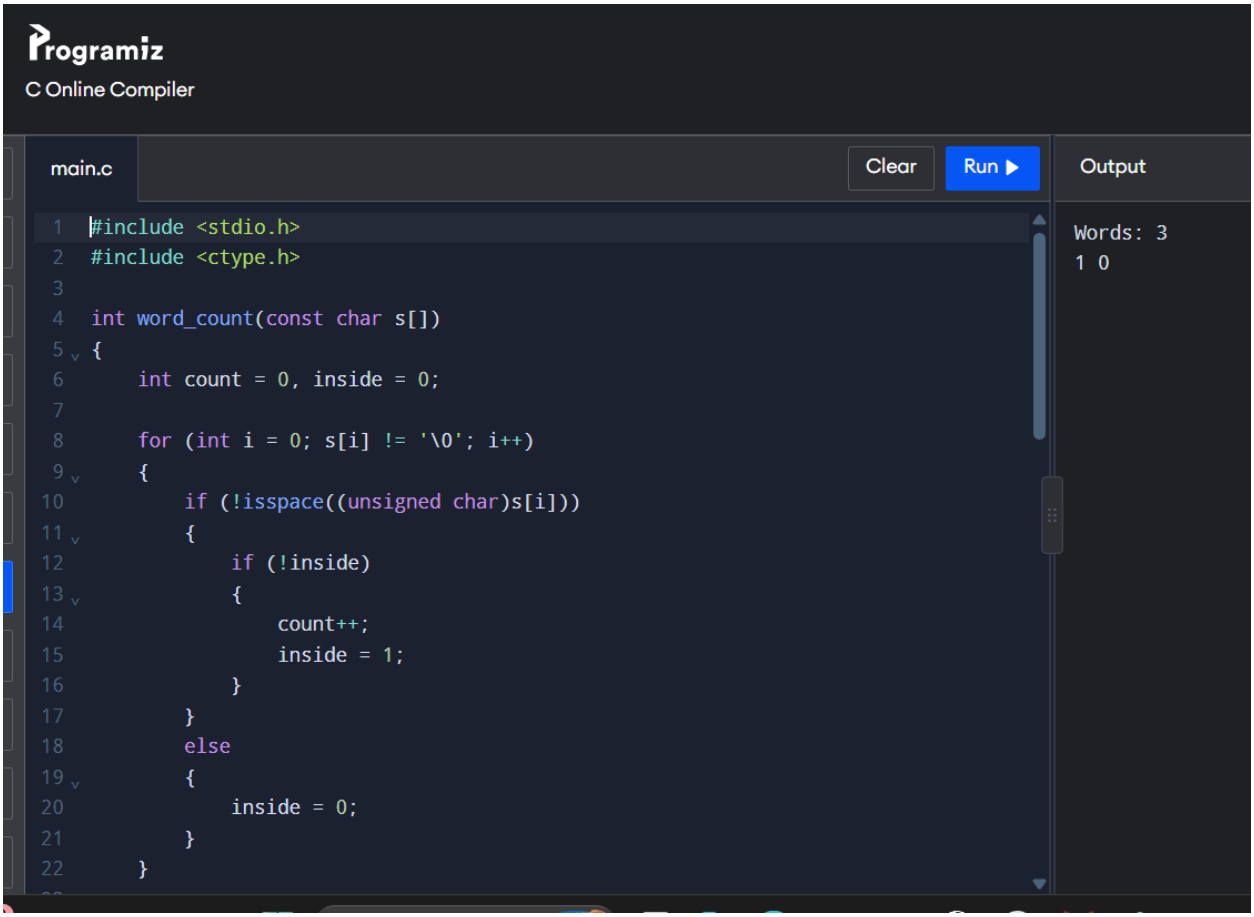
```
{
   int left = 0, right = 0;
   while (s[right] != '\0') right++;
   right--;
   while (left < right) {
      while (left < right && !isalpha((unsigned char)s[left]))  left++;
      while (left < right && !isalpha((unsigned char)s[right])) right--;
      if (tolower((unsigned char)s[left]) != tolower((unsigned char)s[right])) return 0;
      left++; right--;
   }
   return 1;
}
```

Output (matches expected):

```
Words: 7
1 0
```

Other tests: empty message → 0 words and palindrome result 1 (an empty string reads the same both ways, a convention that should be documented); "hello world" → upper-case HELLO WORLD → reversed DLROW OLLEH. The same functions written with strlen, strcpy, strcmp and toupper gave identical results (e.g. toupper version of "Level" → LEVEL, strlen 5, strcmp of two equal strings 0).

Tab test: word_count("go\tnow") returned 1 with the space-only test, because a tab is a non-space character, so "go\tnow" looks like one long word. Replacing s[i] != ' ' with !isspace((unsigned char)s[i]) (from ctype.h) treats tabs and newlines as separators and returns 2.



Student – Character-Level Design

- **Role of '\0':** a C string is a char array that ends with the null character (value 0). It is how every string function knows where the text stops, since the array has no stored length. The array must have room for it: "Meet" needs 5 bytes (measured sizeof = 5).

```
COUNT_WORDS(s):  count = 0 ; inside = 0
```

```
  for each character ch of s until '\0':
     if ch is not a space:
        if inside == 0: count = count + 1 ; inside = 1
     else: inside = 0
  return count          /* several spaces simply keep inside = 0 */
```

- **Why not recompute the length in every iteration:** strlen scans the whole string every time, O(n) per call, so a loop that calls it in its condition costs O(n²). Measured for n = 200,000 characters at -O0: 379.5 ms with strlen in the condition against 0.63 ms with the length stored once (about 600× faster).
- **char s[] = "OK" versus char *s = "OK":** the first is a 3-byte array in writable memory, initialised from the literal (sizeof 3), so s[0] = 'o' is legal. The second is a pointer (sizeof 8 on this system) to a string literal that may sit in read-only memory, so modifying it is undefined behaviour and usually crashes.

Student – Library Functions and Safety

strcpy overflow: copying a 44-character message into char small[8] gives, at compile time, "warning: writing 45 bytes into a region of size 8 overflows the destination" and, at run time:

```
*** stack smashing detected ***: terminated
Aborted    (exit status 134)
```

Without the compiler's protection the extra bytes would overwrite neighbouring variables and the return address, a classic security hole. Safe replacements, tested with the same message (output: "This me", 7 characters plus the terminator):

```
strncpy(small, msg, sizeof small - 1); small[sizeof small - 1] = '\0';   /* must terminate manually */
snprintf(small, sizeof small, "%s", msg);                          /* always terminates */
fgets(buf, sizeof buf, stdin); buf[strcspn(buf, "\n")] = 0;          /* bounded input, strip newline */
```

Why gets() was removed: gets() cannot be told the size of the buffer, so any input longer than the array overflows it and the program cannot defend itself. It was deprecated in C99 and removed in C11; fgets takes a size argument and replaces it.

Joint conclusion: write your own function when learning, when the task needs a rule the library lacks (words separated by several spaces, ignoring non-letters in a palindrome) or when no library exists (some embedded systems). Rely on string.h for standard operations (copy, compare, length): it is tested, optimised and consistent – but only with bounded versions and buffers of known size.

Alternative Concept and Complexity Analysis

for (i = 0; i < strlen(s); i++) calls strlen n + 1 times, each O(n): O(n²) in total (379.5 ms against 0.63 ms for n = 200,000). Storing the length first, or testing s[i] != '\0', is O(n). (An optimising compiler may hoist strlen out of the loop when it can prove the string does not change, but the code should not rely on that.) A recursive reversal – swap s[i] and s[j], then call itself for i + 1, j − 1 – is also O(n) time but uses n/2 + 1 stack frames (measured depth 6 for 10 characters), i.e. O(n) stack space against O(1) for the loop; at a few dozen bytes per frame a string of a million characters could exhaust the default 8 MB stack. For SMS-length messages (160 characters) both are fine; the loop is safer.

Q6 – A Directory of Names: Sorting and Searching Strings

```
#define LEN 30
char names[500][LEN];
for (int i = 0; i < n; i++) {
   if (!fgets(names[i], LEN, stdin)) break;
   names[i][strcspn(names[i], "\n")] = '\0';       /* remove the newline */
}
/* sort: insertion sort on names with strcmp and strcpy */
char tmp[LEN];
for (int i = 1; i < n; i++) {
   strcpy(tmp, names[i]); int j = i - 1;
   while (j >= 0 && strcmp(names[j], tmp) > 0) { strcpy(names[j+1], names[j]); j--; }
   strcpy(names[j+1], tmp);
}
/* binary search on names with strcmp; duplicates: names[i] and names[i+1] equal */
```

Student – Measurement and Improvement

Random names of 5–19 letters (about 13 bytes copied each), insertion sort on copies versus insertion sort on an index array (strcmp calls are the same):

n	strcmp calls	strcpy calls (copy sort)	Bytes copied (copy sort)	Index sort: int moves (4 bytes each)
100	2,486	2,587	32,262	2,389
500	61,289	61,795	810,177	60,797
5,000	6,239,453	6,244,457	81,405,096	6,234,459

Sorting indices does the same comparisons but replaces each 13-byte string copy (a function call plus a loop over the characters) with one 4-byte integer assignment – roughly 80 MB of copying avoided at n = 5,000.

Names of length 29 and 30 in names[][30]: fgets(names[i], 30, …) reads at most 29 characters plus the terminator. A 29-letter line leaves its newline unread, so the next fgets returns an empty name (measured: names[1] = "", length 0). A 30-letter line is truncated to 29 characters and its last letter and newline stay in the input, corrupting the next entry. So the longest storable name is 29 characters, and the input routine should read into a larger buffer (e.g. 64 bytes), check for the newline, and report too-long names instead of splitting them.

Binary search, n = 500: average 8.00 comparisons for present names and 9 for a missing name ($\lfloor \log_2 500 \rfloor + 1 = 9$).

Student – Design and Correctness

- **strcmp:** compares the two strings character by character as unsigned codes and returns 0 if equal, a negative value if the first is smaller at the first difference, and a positive value if larger. "apple" versus "Apple": the first letters are 'a' (97) and 'A' (65), so strcmp returns a positive number (measured: 1) and "Apple" sorts before "apple", with all upper-case names before all lower-case names.
- **Case-insensitive comparison:** compare tolower() of each pair of characters: int ci_cmp(const char *a, const char *b){ while (*a && *b) { int d = tolower((unsigned char)*a) − tolower((unsigned char)*b); if (d) return d; a++; b++; } return tolower((unsigned char)*a) − tolower((unsigned char)*b); } (ci_cmp("apple", "Apple") = 0 measured). strcasecmp is a POSIX alternative.
- **Cost of swapping two names:** a swap through a temporary is three strcpy calls, about 3 × (L + 1) bytes (up to 90 bytes for L = 29), against three integer assignments (12 bytes) for an index swap.
- **Duplicates:** equal strings compare equal, so after sorting they are adjacent; one pass comparing names[i] with names[i + 1] finds them in O(n). (For case-insensitive duplicates, sort with ci_cmp.)

Joint conclusion – why sort indices or pointers: each swap or shift moves 4–8 bytes instead of up to 30, with no copy loop. The trade-off is O(n) extra memory for the index array (2 KB for 500 names), an extra indirection (names[idx[j]]) on every comparison, and the names remain in their original positions, so every use of the sorted list goes through the index.

Alternative Concept and Complexity Analysis

```
char *list[500];
for (int i = 0; i < n; i++) list[i] = names[i];
static int cmp(const void *a, const void *b) { return strcmp(*(char * const *)a, *(char * const *)b); }
qsort(list, n, sizeof(char *), cmp);
```

qsort needs about n log2 n comparisons (measured 3,870 strcmp calls for n = 500, against 61,289 for the insertion sort on copied names), each costing up to L characters, so O(L · n log n) time. Each swap moves one 8-byte pointer instead of copying up to 30 bytes. Extra space: the pointer array of 500 × 8 = 4,000 bytes. As in the index sort, the names themselves never move.

Q7 – Design an Election Counting Program from First Principles

Stage 1 – Interpret the Problem (Joint)

- **Input:** c ≤ 12 candidates; up to V = 5,000 integer votes. Rule: a vote x is valid only if 1 ≤ x ≤ c; anything else (0, negative, above c) is invalid.
- **Output:** number of valid and invalid votes; the votes for each candidate in descending order; the winner; whether there is a tie.
- **Arrays:** votes[5000] (20,000 bytes; not needed if votes are processed as they arrive), count[13] (indices 1..12; index 0 unused), results[12] of (candidate, votes) pairs.

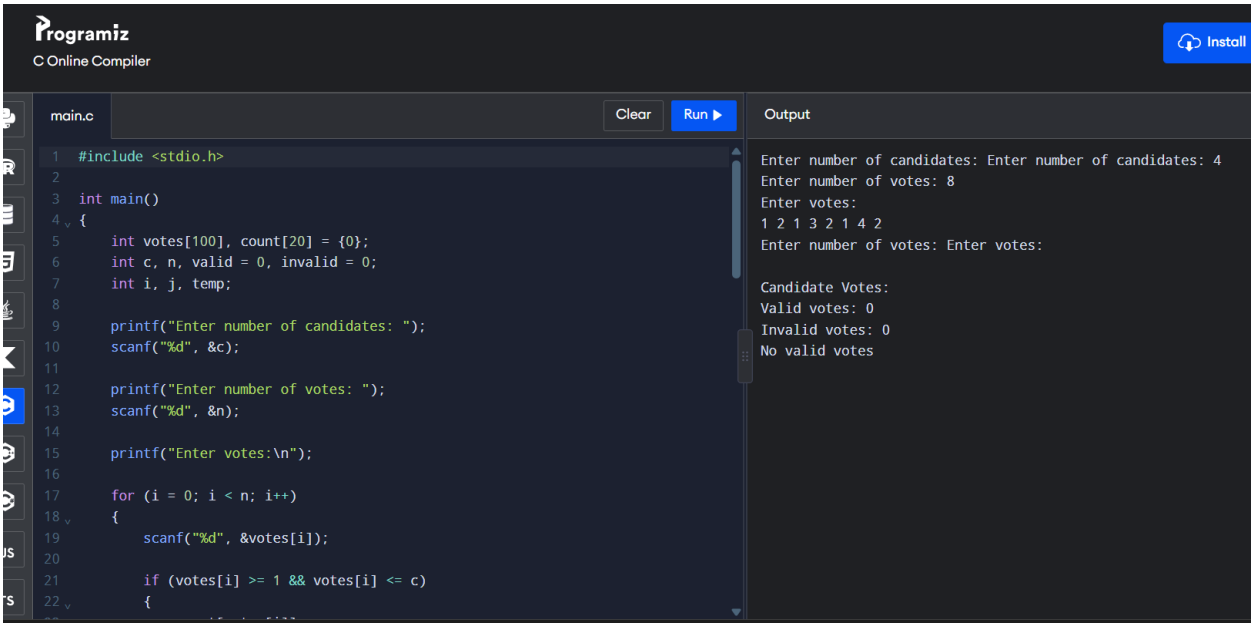
Measurable requirements:

- R1 valid + invalid = number of votes entered.
- R2 the sum of all candidate counts = valid votes.
- R3 the printed list is in non-increasing order of votes.
- R4 the winner's count is the maximum; a tie is reported exactly when two or more candidates share the maximum.
- R5 one pass over the votes: time O(V + c log c), with c ≤ 12 and V ≤ 5,000 taking well under a millisecond (measured below).

Stage 2 – Design A

```
COUNT(votes, V, c):
  count[1..c] = 0 ; valid = 0 ; invalid = 0
  for each vote x in votes:
    if 1 <= x <= c: count[x] = count[x] + 1 ; valid = valid + 1
    else:           invalid = invalid + 1
  build results[i] = (i, count[i]) for i = 1..c
  sort results by votes descending (equal votes: lower candidate number first)
  if valid == 0: report "no valid votes"
  tied = 1
  while tied < c and results[tied].votes == results[0].votes: tied = tied + 1
  if tied > 1: report "TIE between", results[0..tied-1]
  else:        report "winner", results[0]
```

Tie detection: after sorting, all candidates with the maximum are at the front, so a tie exists exactly when the second candidate has the same count as the first; counting how many share it names the tied candidates. Sorting 12 results costs a negligible O(c log c) (any simple sort will do).



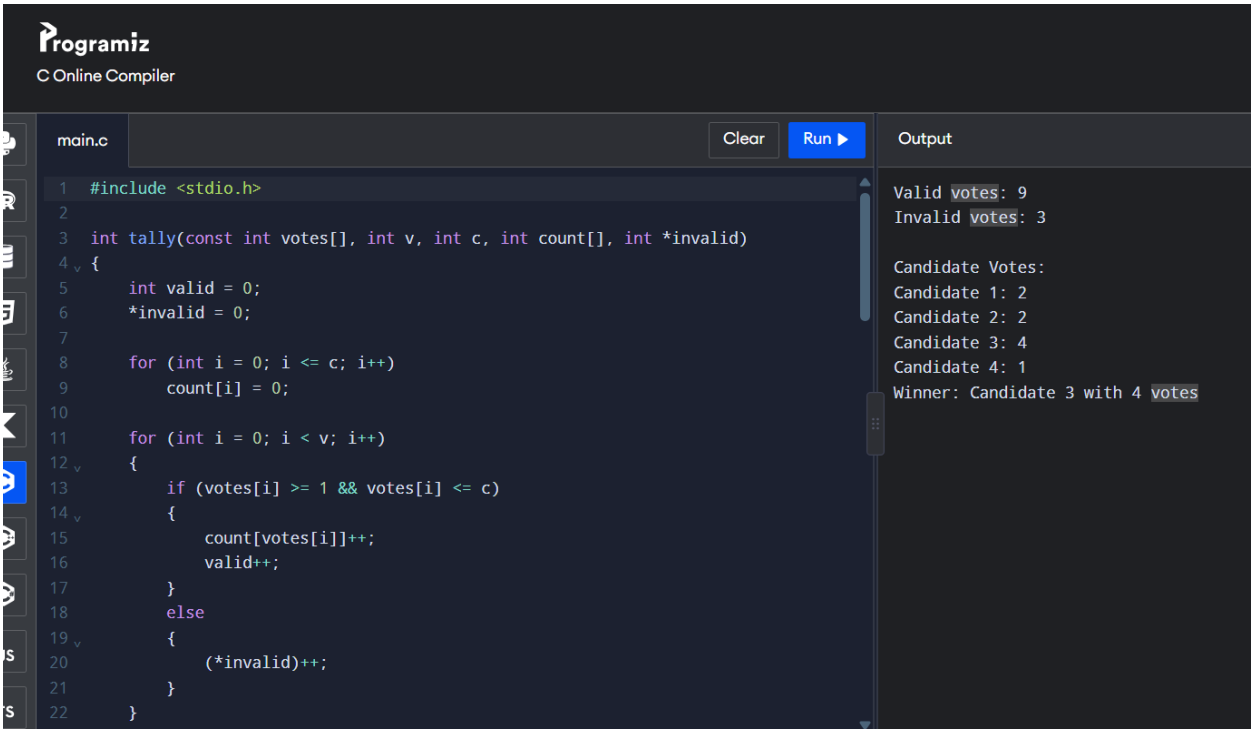
Stage 3 – Implement and Measure (B)

```
int tally(const int votes[], int v, int c, int count[], int *invalid)
{
   int valid = 0; *invalid = 0;
   for (int i = 0; i <= c; i++) count[i] = 0;
   for (int i = 0; i < v; i++) {
      if (votes[i] >= 1 && votes[i] <= c) { count[votes[i]]++; valid++; }
      else (*invalid)++;
   }
   return valid;
}   /* results sorted with qsort and a comparator: more votes first, then lower number */
```

Test	Votes	Output	Hand check
Invalid votes (c = 4)	1 2 2 3 0 3 3 4 −2 5 1 3	valid = 9, invalid = 3; cand 3: 4, cand 1: 2, cand 2: 2, cand 4: 1; winner 3	3 appears 4 times; 0, −2, 5 invalid ✓
Tie (c = 2)	1 2 1 2 9	valid = 4, invalid = 1; TIE between 1 and 2 with 2 votes	✓
All invalid (c = 3)	0 −1 7	valid = 0, invalid = 3; "no valid votes"	✓

Votes	Count array (measured)	Search a name list for every vote	Sort all votes, count runs
5,000	0.012 ms	0.444 ms (28,140 strcmp)	0.421 ms
500,000	2.88 ms	51.9 ms (2,785,180 strcmp)	52.3 ms

Times are gcc -O0 on my machine (averaged over repeated runs; yours will differ). The count array took 2.4 ns per vote at 5,000 votes and 5.8 ns per vote at 500,000 – close to linear; the small increase comes from memory effects. For 500,000 votes the count array was about 18 times faster than the name search and the sort-based method.



Stage 4 – Defend the Design (Joint)

- **Why a count array:** the candidate number is itself the index, so each vote is counted in one step, O(1), with no searching. Searching a list of names for every vote costs up to c string comparisons per vote (O(v·c); 2.8 million strcmp calls for 500,000 votes) and needs string comparison instead of a single integer test.
- **For 500 candidates:** count[501] still works unchanged – counting remains O(V) and memory is only 2 KB; the sort of results grows to O(c log c) with c = 500, still trivial; the name search would be 40 times slower. The report would probably show only the top few candidates plus the tie check, and MAXC and the result arrays would be enlarged or allocated dynamically. If candidate numbers were not small consecutive integers (e.g. 6-digit IDs), a hash table or a mapping from ID to index would be needed.
- **A remaining limitation:** the program trusts that one entry is one voter. It cannot detect duplicate voting, does not record why a vote was invalid, and relies on a fixed upper limit on candidates and dense numeric candidate IDs.

Alternative Concept and Complexity Analysis

Method	Time (v votes, c candidates)	Space	Measured at v = 500,000, c = 12
Count array indexed by candidate	O(v + c)	O(c) (plus O(v) only if the votes are stored)	2.88 ms
Search a list of names for each vote	O(v · c)	O(c)	51.9 ms
Sort all votes, count runs of equal values	O(v log v)	O(v) to hold the votes	52.3 ms

The count array is best whenever the categories are a small, dense range of integers. The name search is the natural way when only names are available but scales badly with c; sorting is attractive only when the categories are unbounded or not numeric.

Feedback and Reflection;

This assignment was designed as a group project, but I completed it individually. Doing so gave me full ownership of every task, from diagnosing the poor data design to implementing the canteen system, and deepened my understanding of structures, unions, storage classes and memory layout. The main limitation was the increased workload and the absence of peer review, which a group would have provided. In future, I would welcome a group version to benefit from shared ideas and code reviews.